

JSON++: Making Configuration Reuse Declarative

Hiroshi Ukai
LegalOn Technologies
Shibuya, Tokyo

hiroshi.ukai@legalontech.jp



Abstract—Configuration files are widely used to describe infrastructure, deployment environments, and application behavior. As systems grow, these configurations often accumulate duplicated fragments across services and environments, making them difficult to maintain and evolve. Existing approaches address this problem through templating languages or environment overlays, but these mechanisms often introduce additional complexity or require developers to adopt new configuration languages.

This article introduces *JSON++*, a lightweight extension of JSON that enables declarative structural reuse while preserving compatibility with existing JSON tooling. JSON++ provides two minimal abstractions: inheritance for reusing configuration structures and expression evaluation for computing values dynamically. We present *jq++*, a tool that evaluates JSON++ configurations and produces ordinary JSON. Because *jq++* accepts JSON-compatible input and emits standard JSON, it can be inserted transparently into existing configuration pipelines without modifying either upstream generators or downstream systems.

Our experience suggests that small, compatible extensions to widely adopted data formats can significantly improve configuration reuse while avoiding the complexity of new configuration languages.

1 INTRODUCTION — THE PROBLEM OF CONFIGURATION DUPLICATION

Modern software systems rely heavily on configuration files to describe infrastructure, deployment environments, and application behavior. Configuration artifacts appear in many forms, including Kubernetes manifests, CI/CD pipelines, infrastructure definitions, and application runtime settings. These artifacts are commonly written in formats such as JSON [1], YAML [2], or HCL [3] and processed by a wide range of tools across the software delivery pipeline.

Although these formats are expressive enough to represent structured data, they provide limited mechanisms for structural reuse. As systems grow, engineers frequently duplicate configuration fragments across services, environments, and repositories. Small changes must then be propagated manually, increasing maintenance effort and introducing the risk of inconsistencies.

The problem becomes particularly visible in modern infrastructure systems. For example, Kubernetes deployments often share runtime settings across multiple workloads, while environment overlays capture differences between

development, staging, and production environments. Existing mechanisms such as overlays, templating engines, and application-specific configuration languages partially address this problem but often introduce additional complexity. Developers must reason not only about the resulting configuration but also about the template logic or language semantics used to generate it.

Several tools address configuration duplication by introducing new configuration languages such as Jsonnet [4] and CUE [5]. While expressive, these approaches require developers to adopt additional syntax and toolchains beyond JSON.

In this article we explore a different approach. Instead of introducing a new configuration language, we extend JSON itself with a small number of declarative mechanisms for structural reuse. The resulting notation, called *JSON++*, preserves full compatibility with JSON while enabling configurations to inherit structures and compute values dynamically.

Unlike systems such as Jsonnet or CUE, which introduce new configuration languages, JSON++ keeps JSON itself as the authoring language and adds reuse mechanisms through lightweight annotations that remain valid JSON syntax.

JSON++ introduces two minimal abstractions. First, configurations may extend other configurations using an inheritance-like mechanism that merges structures while allowing selective overrides. Second, configuration values may be defined through evaluable expressions, allowing derived values to be computed directly within the configuration. Together these mechanisms make it possible to describe complex configuration relationships concisely while retaining compatibility with existing JSON tooling.

To demonstrate the practicality of this approach we present *jq++*, a tool that evaluates JSON++ configurations and produces ordinary JSON as output. Because both the input and output remain JSON-compatible, *jq++* can be integrated transparently into existing configuration pipelines. Editors, validators, and deployment systems continue to operate on standard JSON without modification.

Contributions

This article makes three main contributions.

- We identify a practical limitation in widely used configuration formats: the lack of mechanisms for expressing structural reuse directly within configuration data.
- We introduce JSON++, a lightweight extension of JSON that provides declarative inheritance and expression evaluation while preserving compatibility with the JSON data model.
- We present jq++, a tool that implements the JSON++ evaluation model and integrates into existing configuration pipelines by producing ordinary JSON as output.

The remainder of this article is organized as follows. Section 2 presents a motivating example illustrating the limitations of existing configuration reuse mechanisms. Section 3 describes the design principles behind JSON++. Section 4 introduces the architecture of jq++ and its evaluation pipeline. Section 5 discusses implementation experiences and practical lessons from building the tool. Section 6 shows how JSON++ integrates with existing configuration ecosystems. Section 7 and Section 8 summarize observations and lessons learned from applying the approach in practice. Finally, Section 9 discusses related work and Section 10 concludes.

2 A MOTIVATING EXAMPLE FROM KUBERNETES CONFIGURATION

Configuration duplication becomes visible quickly in systems that are deployed across multiple environments. Consider a web application deployed to Kubernetes in three stages: development, staging, and production. The system consists of five workloads: a backend service, an admin service, a scheduled job service, a frontend, and a database.

A common organization is to maintain a base directory and separate overlays for each environment, a structure widely used in Kubernetes [6], [7] configuration repositories and tools such as Kustomize[8].

As illustrated in Figure 1, several workloads often share identical runtime configuration fragments. Listing 2 shows a typical base-and-overlays structure used with Kustomize.¹

Listing 1. Typical base-and-overlays structure used with Kustomize

```
base/
  backend.yaml
  admin.yaml
  job.yaml
  frontend.yaml
  db.yaml

overlays/
  dev/
    kustomization.yaml
  stg/
    kustomization.yaml
  prd/
    kustomization.yaml
```

1. Examples are shown in YAML for readability. YAML documents are typically convertible to JSON data structures, and many systems internally operate on JSON-like data models. Because JSON++ operates on the JSON data model, these examples can be interpreted equivalently as JSON.

This structure captures differences between environments. For example, the production overlay may increase replica counts or storage sizes. However, many configuration fragments are shared not only across environments but across related workloads within the same system.

For instance, the backend service, admin service, and scheduled job service often share the same runtime configuration such as database connectivity and cache endpoints. The backend and admin services may additionally share HTTP health probes. As a result, similar configuration fragments appear repeatedly across multiple manifests.

Listing 2. Repeated runtime configuration across multiple workloads

```
# backend.yaml
containers:
  - name: backend
    image: example/backend:1.0
    env:
      - name: DB_HOST
        value: postgres
      - name: DB_PORT
        value: "5432"
      - name: REDIS_HOST
        value: redis
      - name: LOG_LEVEL
        value: info
    livenessProbe:
      httpGet:
        path: /health
        port: 8080

# admin.yaml
containers:
  - name: admin
    image: example/admin:1.0
    env:
      - name: DB_HOST
        value: postgres
      - name: DB_PORT
        value: "5432"
      - name: REDIS_HOST
        value: redis
      - name: LOG_LEVEL
        value: info
    livenessProbe:
      httpGet:
        path: /health
        port: 8080
```

The base-and-overlays model captures differences across environments, but it does not naturally capture structural similarities among related workloads.

Template systems reduce duplication through parameterization but often obscure the final configuration structure, making debugging more difficult.

Figure 2 illustrates this point: configuration reuse typically spans both environment-specific variation and similarities among related workloads.

The motivating problem is therefore not merely duplication across dev, stg, and prd. Real systems contain multiple overlapping reuse relationships, including environment variation and workload similarity. A practical configuration mechanism should support these relationships directly while remaining compatible with existing tooling.

This observation motivates the design of JSON++, which introduces lightweight mechanisms for expressing structural reuse directly within JSON-compatible configuration data.

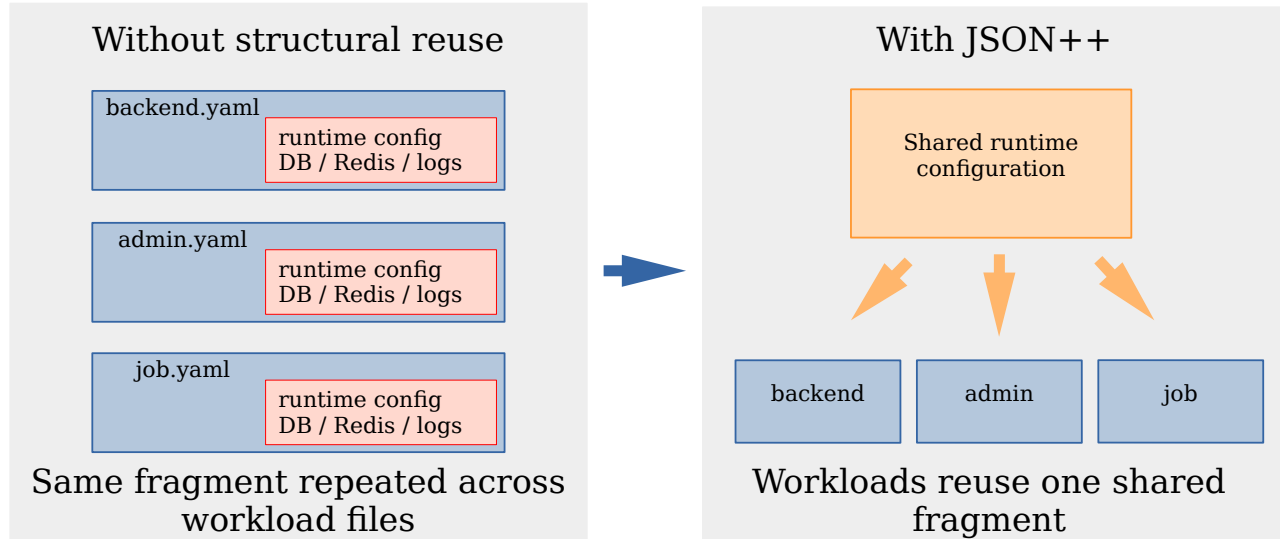


Fig. 1. Configuration duplication across related workloads. Without structural reuse, the same runtime configuration fragment is repeated in multiple workload files. With JSON++/jq++, the shared fragment can be factored out and reused declaratively.

		Environment-specific variation		
		dev	stg	prd
Workload similarity	backend	env probe db	env probe db	env probe db
	admin	env probe db	env probe db	env probe db
	job	env db	env db	env db

Fig. 2. configuration reuse spans multiple independent dimensions. Columns represent environment-specific variation, while rows represent similarities among related workloads. In practice, duplication arises across both axes rather than along only one hierarchy.

3 DESIGN PRINCIPLES BEHIND JSON++

JSON++ was designed around three principles that emphasize compatibility and simplicity.

Principle 1 — Expressive completeness

Expressive completeness: Any JSON can be represented as JSON++. An important property of JSON++ is that it does not restrict the space of configurations that can be produced. The jq++ evaluation function maps JSON++ documents to ordinary JSON documents. For any valid JSON document J , there exists a JSON++ document X such that $\text{jq++}(X) = J$. In other words, the evaluation function is surjective onto JSON. This property ensures that JSON++ can generate any configuration expected by downstream systems, allowing jq++ to be introduced as a preprocessing step without reducing expressiveness.

In practice, this means that adopting JSON++ never forces downstream tools to change their expectations about the resulting configuration structure².

Syntax compatibility: All JSON++ documents are valid JSON. Moreover, JSON++ documents remain valid JSON, which allows existing parsers and tooling to process them without modification.

Principle 2 — JSON Data-Model Compatibility

JSON++ operates at the level of the JSON data model rather than a specific surface syntax. Because many configuration languages such as YAML, HCL, and TOML are routinely translated into JSON during processing, the same reuse mechanisms can be applied to configurations written in those formats. In practice this means that JSON++ concepts naturally extend to JSON-compatible representations such as YAML without requiring changes to the authoring language.

This also implies that the same extension concept naturally applies to other JSON-compatible formats such as YAML.

Principle 3 — Declarative Structural Reuse

JSON++ introduces explicit constructs for expressing structural reuse. Configurations can extend other configurations and override selected fields. This inheritance-like mechanism enables developers to describe relationships between configurations directly in the data structure rather than through external templating logic.

2. All JSON++ meta-keywords are escapable

jq++ Evaluation Pipeline

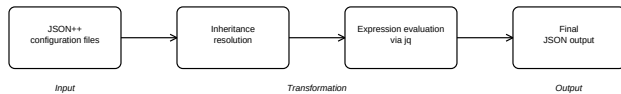


Fig. 3. jq++ evaluation pipeline. jq++ reads JSON++ configuration files, resolves inheritance, evaluates expressions using jq, and produces ordinary JSON that can be consumed by existing systems.

4 ARCHITECTURE OF JQ++

The jq++ tool implements the JSON++ evaluation model and produces ordinary JSON as its output. This design allows JSON++ configurations to be processed as a pre-processing step before being passed to other tools.

Figure 3 illustrates the evaluation pipeline implemented by jq++. The pipeline transforms JSON++ input into standard JSON through a sequence of deterministic processing stages.

The pipeline begins with a JSON++ configuration file.

After parsing, jq++ constructs an internal representation in which JSON++ constructs such as inheritance directives remain unresolved.

The next stage resolves structural reuse relationships. Configurations can extend other configurations through explicit inheritance directives. jq++ loads referenced configuration files, merges their contents, and constructs a combined configuration object. Overrides defined in the extending configuration replace or augment fields inherited from parent configurations.

After inheritance resolution, jq++ evaluates expressions embedded in the configuration. Expression evaluation relies on the jq language [9], which provides powerful data transformation capabilities. jq expressions can define reusable helper functions that encode common configuration rules. By reusing jq’s evaluation model, jq++ avoids introducing a new expression language while allowing configurations to compute values dynamically.

Once evaluation completes, jq++ produces a fully expanded configuration. All inheritance relationships have been resolved and all expressions have been evaluated. The resulting document is standard JSON and can therefore be consumed directly by downstream tools.

This architecture separates configuration reuse from the systems that consume the resulting configuration. Existing tools continue to operate on ordinary JSON, while jq++ acts as a pre-processing stage that introduces reusable configuration abstractions.

To ensure predictable behavior, jq++ resolves inheritance in a deterministic order. Parent configurations are expanded before their children, and multiple parents are merged according to the explicit order specified in the `$extends` directive. Cyclic inheritance and reference relationships are detected and rejected during evaluation.

5 IMPLEMENTATION EXPERIENCE

The jq++ tool is implemented in Go and builds upon the widely used jq processing model. The implementation focuses on predictable evaluation and compatibility with existing workflows. Special attention was given to deterministic resolution of inheritance relationships and to transparent error reporting, including detection of cyclic inheritance relationships. These properties are essential when configurations become large and are maintained by multiple teams.

The jq++ implementation is available as open source at <https://github.com/dakusui/jqplusplus>, together with documentation and usage examples.

Also, during implementation we observed that configuration reuse requires two distinct composition mechanisms. The first mechanism is inheritance, represented by the `$extends` directive, which allows a configuration to derive from one or more base definitions. However, practical configuration pipelines often involve a second scenario in which external fragments are provided by users or by deployment environments. In these cases the goal is not to derive a new configuration from a base definition, but to inject additional structure that may override or complete existing fields. To support this scenario we introduced a second directive, `$includes`.

While `$extends` models inheritance relationships between configuration definitions provided by the system itself, `$includes` enables composition of configuration fragments supplied from external sources.

This distinction proved important in systems where platform defaults are extended by team-specific overrides.

Another practical lesson emerged from the need to resolve configuration files across large repositories.

Many configuration systems require imported files to be specified using relative paths. While simple, this approach becomes inconvenient as configuration collections grow and are shared across projects.

jq++ therefore introduces a deterministic search-path mechanism controlled by the environment variable `JF_PATH`.

The mechanism is conceptually similar to environment-based search paths used in systems such as `PATH`, `PYTHONPATH`, or `CLASSPATH`. Referenced configuration files are resolved by scanning the directories listed in `JF_PATH` in a fixed order.

Without deterministic resolution rules, configuration inheritance can become difficult to reason about, particularly in large repositories maintained by multiple teams.

A remaining implementation challenge is source-level diagnosability. Because jq++ resolves inheritance, inclusion, and expression evaluation across multiple JSON++ files before emitting a single effective JSON document, debugging failures can require tracing an erroneous value in the expanded output back to the original source fragment that introduced it. In practice, this suggests the need for provenance-aware evaluation, where generated nodes retain references to their originating files and paths. Improving source-aware diagnostics is an important direction for future tooling, particularly for large configuration repositories where the effective JSON document may be derived from dozens of input fragments.

6 USING JSON++ WITH EXISTING ECOSYSTEMS

An important property of JSON++ is that it integrates naturally with configuration ecosystems that already rely on JSON as an underlying data model. Many widely used configuration languages, including YAML, HCL, and TOML [10], are routinely converted into JSON during processing. Because YAML is a superset representation of JSON, the same idea can be viewed from the opposite direction as well. In practice, YAML documents interpreted under the JSON++ evaluation model effectively form a YAML++ notation. This observation highlights that JSON++ does not introduce a new configuration language but instead extends the JSON data model itself. Any language that can be mapped to JSON can benefit from the same reuse mechanisms. As a result, mechanisms defined for JSON can be applied to configurations written in these formats without modifying the downstream tools.

6.1 Revisiting the Motivating Example

Section 2 illustrated a common Kubernetes configuration pattern in which multiple workloads share similar runtime configuration. In the example, the backend service, admin service, and scheduled job service repeat identical environment variables and health probes.

The base-and-overlays model used by tools such as Kustomize is effective for representing differences across environments (e.g., dev, staging, and production). However, it does not directly address structural reuse among related workloads within the same environment. As a result, configuration fragments such as database connectivity, cache endpoints, or health probes are often duplicated across multiple manifests.

JSON++ addresses this limitation by allowing configurations to inherit shared fragments directly. Instead of repeating identical structures, developers can factor them out into reusable definitions and extend them where needed.

6.2 Example: Structuring System Configuration

The motivating example in Section 2 illustrated how similar configuration fragments are repeated across multiple workloads. While inheritance alone can remove duplicated structures, real systems also contain relationships among components and environment-dependent values.

Consider a system consisting of four components: a backend service, an admin service, a frontend, and a database. In many configurations, connection endpoints and port numbers are written directly inside component definitions. This practice scatters operational facts across the configuration and makes it difficult to change them consistently.

Using JSON++, these operational facts can be separated from component definitions. Ports and environment-specific URLs can be defined in reusable maps and included into the configuration.

Listing 3. System configuration using JSON++ abstractions

```
$local:
  ports:
    $includes: ports.yaml
```

```
urls:
  $includes: urls.yaml

base:
  port: eval:port_of(type())
  image: eval:image_of(type())

components:
  backend:
    $extends: [base]
    type: backend
    databaseUrl: eval:url_of("database")

  admin:
    $extends: [base]
    type: admin
    databaseUrl: eval:url_of("database")

  frontend:
    $extends: [base]
    type: frontend
    backendUrl: eval:url_of("backend")

database:
  $extends: [base]
  type: database
```

In this representation, component blocks describe the logical topology of the system rather than concrete connection values. Operational facts such as ports and URLs are defined once and referenced through functions such as `port_of(...)` and `url_of(...)`.

Environment differences can be isolated by providing different definitions of the URL map. jq++ resolves included files through the deterministic search path controlled by the `JF_PATH` environment variable. Switching environments therefore requires replacing only the environment-specific file while leaving the main configuration unchanged.

As reusable fragments and evaluation functions accumulate, the configuration repository effectively becomes a small domain-specific language for describing system deployments, while remaining entirely within the JSON ecosystem.

6.3 Integration into Existing Toolchains

This workflow enables an incremental adoption path. Engineers can continue writing configuration files in their preferred notation while introducing JSON++ only where structural reuse becomes beneficial.

For example, Kubernetes manifests written in YAML can be converted to JSON, processed with jq++, and then supplied to the Kubernetes API server in their standard form:

YAML → JSON → jq++ → JSON → Kubernetes

Because the output remains ordinary JSON, editors, validators, linters, and deployment systems continue to operate unchanged. jq++ acts purely as a preprocessing stage that resolves inheritance relationships and evaluates expressions before the configuration is consumed by downstream tools.

This approach allows organizations to improve configuration reuse without migrating to a new configuration language. Existing configuration files remain valid and understandable, while JSON++ provides additional expressive power where needed.

7 EXPERIENCE AND OBSERVATIONS

Our experience developing jq++ and applying JSON++ to real configuration repositories revealed several practical observations.

7.1 Reuse Relationships Are Multi-Dimensional

In practice configuration reuse arises along multiple dimensions such as environment differences and workload similarity. For example, Kubernetes deployments often reuse configuration fragments both across environments (e.g., development and production) and across related services within the same environment. JSON++ proved particularly useful in situations where these reuse relationships overlap.

7.2 Two Distinct Composition Scenarios

During implementation we discovered that configuration composition occurs in two fundamentally different scenarios. The first scenario corresponds to derivation, where a configuration is defined by extending a base definition. This scenario is naturally modeled by the `$extends` directive. The second scenario corresponds to augmentation, where external fragments are injected into an existing structure. This scenario motivated the introduction of the `$includes` directive. Distinguishing these two mechanisms simplified the mental model for users and reduced ambiguity in merge semantics.

7.3 Local Reuse Is as Important as File-Level Reuse

Early prototypes focused primarily on file-level inheritance. However, practical configurations frequently contain repeated fragments within a single document. Supporting local reusable nodes proved essential for avoiding fragmentation into many small files. The `$local` construct therefore became an important part of the JSON++ design.

7.4 Compatibility Strongly Influences Adoption

Users were significantly more willing to experiment with JSON++ when it preserved compatibility with existing tooling. Because JSON++ documents remain valid JSON and the jq++ output is ordinary JSON, existing editors, validators, and deployment systems required no modification. This property allowed JSON++ to be introduced incrementally within existing configuration pipelines.

7.5 Expression Evaluation Enables Lightweight DSLs

The evaluation mechanism based on jq expressions enabled users to encode simple domain-specific rules directly in configuration data. Examples include deriving port numbers, constructing environment-specific URLs, and generating repeated structures. In practice these computations replaced many ad hoc template fragments.

7.6 Preprocessing Pipelines Fit Existing Workflows

The jq++ preprocessing model integrates naturally into UNIX-style pipelines. Configurations can be generated, transformed, and validated through a sequence of composable tools. This architecture proved easier to integrate into existing CI/CD workflows than approaches that require dedicated configuration runtimes.

7.7 Configuration Repositories Tend to Evolve into DSLs

In several configuration projects we observed that repositories using reusable fragments and evaluation functions gradually evolved into small domain-specific languages.

Initially, configuration files were written as collections of concrete values describing individual components. As reuse mechanisms were introduced, developers began factoring out shared fragments and defining functions that encoded common relationships such as service endpoints, port assignments, and environment-specific identifiers.

Over time, the configuration files increasingly described the logical structure of the system rather than individual values. Developers interacted with the configuration primarily through named abstractions such as functions and reusable definitions. The resulting repository resembled a lightweight domain-specific language for describing system topology and deployment structure.

This evolution occurred incrementally without introducing a new configuration language. Instead, the DSL-like structure emerged from the combination of reusable configuration fragments and limited computation embedded within the configuration data itself (Figure 4).

8 LESSONS LEARNED

The development of JSON++ and the jq++ tool provided several lessons about how configuration abstractions should be designed and introduced into existing ecosystems.

8.1 Reuse Must Reflect Real Configuration Structure

Configuration reuse rarely follows a single hierarchical structure. Instead, real repositories exhibit multiple overlapping reuse relationships such as environment variation, service similarity, and organizational boundaries. Tools designed around a single reuse axis (for example environment overlays) cannot represent these relationships naturally. Allowing inheritance and composition within the configuration structure itself proved more flexible than relying solely on external overlay systems.

8.2 Different Composition Scenarios Require Different Mechanisms

During implementation we observed that configuration composition occurs in two distinct scenarios. One scenario derives a configuration from a base definition, while another augments an existing structure with externally supplied fragments. Modeling both situations with a single mechanism led to ambiguous semantics. Introducing separate directives for derivation (`$extends`) and augmentation (`$includes`) significantly simplified the mental model and clarified merge behavior.

8.3 Local Reuse Is Essential in Large Configurations

Large configuration files often contain repeated fragments within the same document. Early designs focused primarily on reuse across files, but this approach forced developers to split configurations into many small files. Supporting local reusable nodes through the `$local` construct allowed reuse without fragmenting the configuration structure. In practice, local reuse proved as important as file-level reuse.

ports.yaml

```
backend: 8080
admin: 8081
job: 8082
database: 5432
redis: 6379
```

Port-assignment DSL

urls.yaml

```
backend: backend
admin: admin
job: job
database: postgres
redis: redis
```

Endpoint-naming DSL

system.yaml

```
$local:
  ports:
    $includes: [ports.yaml]
  urls:
    $includes: [urls.yaml]
  baseService:
    port: 'eval:port_of(type())'
    databaseUrl: 'eval:url_of("database")'
    cacheUrl: 'eval:url_of("redis")'

  services:
    backend:
      $extends: [baseService]
      type: backend
    admin:
      $extends: [baseService]
      type: admin
    job:
      $extends: [baseService]
      type: job
```

Topology DSL

Fig. 4. Factored configuration fragments can evolve into small domain-specific declarative vocabularies. Here, `ports.yaml` defines port assignments, `urls.yaml` defines logical service endpoint names, and `system.yaml` describes system topology by referring to those abstractions. Rather than merely removing duplication, JSON++ allows repositories to separate concerns into compact, JSON-compatible fragments that resemble lightweight DSLs.

8.4 Lightweight Preprocessors Integrate Well with Existing Pipelines

The `jq++` architecture treats configuration reuse as a preprocessing step that produces ordinary JSON. This design fits naturally into existing UNIX-style pipelines where data flows through a sequence of transformations. Organizations were able to integrate `jq++` into their workflows with minimal disruption because downstream systems remained unchanged. This suggests that lightweight preprocessors are an effective way to extend configuration ecosystems without introducing new runtime environments.

8.5 Expression Evaluation Enables Small Domain-Specific Abstractions

Allowing configuration values to be computed using `jq` expressions enabled users to encode small domain-specific rules directly within configuration files. Typical examples include deriving URLs, generating environment-specific identifiers, and constructing repeated structures.

In practice these simple computations replaced many ad hoc template fragments while keeping configuration files close to their final structure. When reusable fragments are factored by concern, configuration repositories begin to resemble compact declarative vocabularies (Figure 4).

9 RELATED WORK

Several approaches have been proposed to reduce duplication and improve modularity in configuration systems.

Figure 5 summarizes the design space of existing approaches to configuration abstraction.

One class of solutions introduces dedicated configuration or data-templating languages that generate structured configuration data. `Jsonnet` [4] provides a data templating language with functions, inheritance-like mechanisms,

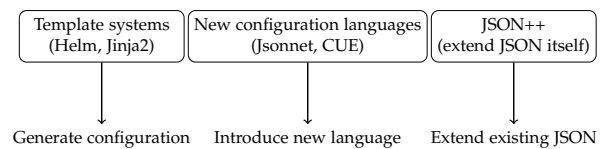


Fig. 5. Three common approaches to configuration abstraction. Template systems generate configuration through embedded template languages, while systems such as `Jsonnet` and `CUE` introduce new configuration languages. `JSON++` takes a different approach by extending JSON itself with lightweight declarative reuse mechanisms.

and imports for constructing configuration objects. `CUE` [5] takes a different approach by combining configuration with constraint validation and schema-like typing, allowing configurations to be expressed as sets of constraints that are unified during evaluation. These systems provide powerful abstraction mechanisms but require users to adopt new languages and toolchains that are distinct from JSON.

From the perspective of `JSON++`, these approaches significantly extend the language surface beyond JSON. In informal terms, the portion of the language outside JSON is substantial. As a result, developers must learn additional syntax and semantics even when their original configuration data is conceptually JSON-like.

Template-based configuration systems follow a similar philosophy. Tools such as `Helm` templates [11], `Jinja2` [12], `Handlebars`, and `ERB` generate configuration files by embedding a template language into the source representation. Although flexible, template-based approaches introduce an additional language layer and require developers to reason about both the template logic and the generated configuration. Also, tools developed for JSON such as IDEs, linters, filters, and others cannot be used for them because they are still not JSON, strictly speaking.

Another widely used strategy is overlay-based com-

position. Kubernetes Kustomize composes configurations by applying patches or overlays to a base configuration, typically to represent differences across environments. This model is effective for managing environment-specific variations but provides limited support for expressing structural reuse among related objects within the same configuration hierarchy.

JSON++ differs from these approaches by preserving JSON as the authoring format and introducing declarative reuse directly within the configuration structure. Rather than defining a new configuration language or embedding a template system, JSON++ adds lightweight annotations that remain compatible with standard JSON syntax. The jq++ tool evaluates these annotations through a preprocessing stage that resolves inheritance relationships and expressions, producing ordinary JSON that can be consumed by existing tools.

This design positions JSON++ as a lightweight structural extension of JSON that complements existing configuration ecosystems while minimizing disruption to established tooling and workflows.

Key Takeaways

- Real configuration repositories contain overlapping reuse relationships, not just environment-specific differences.
- A small extension to JSON can support structural reuse without introducing a new authoring language.
- By emitting ordinary JSON, jq++ fits naturally into existing configuration and deployment pipelines.

10 CONCLUSION

As systems grow in size and complexity [13], configuration repositories often accumulate duplicated fragments that are difficult to maintain and evolve.

This article introduced JSON++, a lightweight mechanism for expressing structural reuse directly within JSON configurations. JSON++ adds two minimal abstractions—inheritance and expression evaluation—while preserving full compatibility with the JSON data model.

We also presented jq++, a tool that evaluates JSON++ configurations and produces ordinary JSON. Because both the input and output remain standard JSON, existing tools such as editors, validators, and deployment systems can continue to operate unchanged. This pre-processing architecture enables organizations to introduce reusable configuration abstractions without replacing existing ecosystems. In practice, this mechanism allows configuration repositories to evolve toward small declarative vocabularies that capture system structure and operational facts while remaining entirely within the JSON ecosystem.

Our experience suggests configuration reuse mechanisms must reflect the multi-dimensional structure of real repositories. By embedding reuse directly in the configuration structure, JSON++ provides a flexible and declarative approach for representing these relationships.

More broadly, JSON++ demonstrates that expressive configuration abstractions can be introduced without inventing new configuration languages. Instead, small and

compatible extensions to widely adopted formats can provide significant practical benefits while minimizing disruption to existing workflows.

Future work will focus on improving source-aware diagnostics and provenance tracking, expanding tooling support, and further exploring how JSON++ can serve as a unifying abstraction layer for JSON-family configuration formats and adjacent domains such as testing and game content definition.

ACKNOWLEDGMENT

The author acknowledges the assistance of OpenAI’s ChatGPT for language editing and structural refinement of the manuscript. All technical ideas, design decisions, and conclusions are the author’s own.

REFERENCES

- [1] T. Bray, “The JavaScript Object Notation (JSON) Data Interchange Format,” IETF RFC 8259, 2017. [Online]. Available: <https://www.rfc-editor.org/rfc/rfc8259>
- [2] O. Ben-Kiki, C. Evans, and I. dot Net, “YAML Ain’t Markup Language (YAML) Version 1.2,” Available at <https://yaml.org/spec/1.2/spec.html>, 2009.
- [3] HashiCorp, “HashiCorp Configuration Language (HCL),” Available at <https://github.com/hashicorp/hcl>, 2014.
- [4] Jsonnet Authors, “Jsonnet specification,” <https://jsonnet.org/ref/spec.html>, accessed 2025-09-11.
- [5] The CUE Authors, “The cue language specification,” <https://cuelang.org/docs/reference/spec/>, 2025, accessed 2025-09-11.
- [6] B. Burns, B. Grant, D. Oppenheimer, E. Brewer, and J. Wilkes, “Borg, omega, and kubernetes,” in *Proceedings of the Tenth European Conference on Computer Systems*, ser. EuroSys ’15. ACM, 2015.
- [7] The Kubernetes Authors, “Kubernetes: Production-grade container orchestration,” <https://kubernetes.io/>, 2015, accessed: 2026-03-19.
- [8] Kubernetes SIG CLI, “Kustomize: Kubernetes native configuration customization,” <https://github.com/kubernetes-sigs/kustomize>, 2018, accessed: 2026-03-19.
- [9] S. Dolan *et al.*, “jq: A lightweight and flexible command-line JSON processor,” Available at <https://stedolan.github.io/jq/>, 2012.
- [10] T. Preston-Werner, “TOML: Tom’s Obvious, Minimal Language,” Available at <https://toml.io/en/>, 2013.
- [11] The Helm Authors, “Helm: The kubernetes package manager,” <https://helm.sh/>, 2015, accessed: 2025-09-10.
- [12] A. Ronacher, “Jinja2: A modern and designer-friendly templating language for python,” <https://palletsprojects.com/p/jinja/>, 2008, accessed: 2025-09-06.
- [13] T. C. CTO, “Working with terraform: 10 months in,” *The Consulting CTO blog*, 10 2017, describes how writing over 40,000 lines of HCL exposed issues of code duplication, low composability, and the difficulty of maintaining Terraform configurations. [Online]. Available: <https://theconsultingcto.com/posts/working-with-terraform-10-months-in/>